

LoRA Training Verification Guide

How to verify that LoRA training with adapter continuity is working correctly in the OpenClaw-RL / simple_rl pipeline.

1. Pre-flight: Check Your Environment

```
# Confirm mlx-tune venv python is the one configured
python3 -c "import simple_rl.config as c; print(c.MLX_TUNE_PYTHON)"
# -> /Volumes/ExternalSSD/train/mlx-tune/.venv/bin/python3

# Confirm that python can import mlx_tune
/Volumes/ExternalSSD/train/mlx-tune/.venv/bin/python3 \
  -c "from mlx_tune import FastLanguageModel, GRPOTrainer; print('ok')"

# Confirm POLICY_MODEL_PATH is set (empty = stub mode, no training)
echo $POLICY_MODEL_PATH
```

2. Run the Integration Smoke Test

Runs 2 real training steps with no W&B, verifies the full pipeline end-to-end.

```
POLICY_MODEL_PATH=mlx-community/Qwen2.5-0.5B-Instruct-4bit \
python3 -m pytest simple_rl/tests/test_mlx_grpo_bridge.py \
  -m integration -v -s
```

A passing test confirms:

- Model loaded from HuggingFace
 - LoRA layers applied via get_peft_model
 - 2 training steps completed without error
 - adapters.safetensors + adapter_config.json written to disk
 - step_losses contains at least one {"step": N, "loss": float} entry
-

3. Verify LoRA Was Applied (stdout)

When grpo_worker.py runs, this block is printed **before the first training step**:

```
=====
LoRA Verification
=====
Enabled           : True
Applied to layers : True
Rank (r)          : 16
Alpha             : 32
Target modules    : ['self_attn.q_proj', 'self_attn.v_proj', ...]
Trainable tensors : 28
```

```

Trainable params : 2,621,440
Frozen params    : 494,032,768
% trainable      : 0.53%

```

Red flags:

What you see	Problem
Trainable params : 0	LoRA did not attach — <code>_apply_lora()</code> failed silently
Applied to layers : False	<code>get_peft_model</code> returned but skipped <code>_apply_lora</code>
% trainable > 5%	Rank very high or too many target modules
LoRA section not printed at all	<code>_log_lora_verification()</code> was skipped

4. Verify Adapter Files on Disk

After a successful round:

```

ls -la logs/grpo_adapters/round_0001/
# adapters.safetensors ← LoRA weights (lora_a, lora_b per layer)
# adapter_config.json   ← rank, alpha, target_modules, fine_tune_type

```

```
cat logs/grpo_adapters/round_0001/adapter_config.json
```

Expected adapter_config.json:

```

{
  "fine_tune_type": "lora",
  "num_layers": 28,
  "lora_parameters": {
    "rank": 16,
    "scale": 2.0,
    "dropout": 0.0,
    "keys": ["self_attn.q_proj", "self_attn.v_proj", "..."]
  }
}

```

The weights file should be several MB (rank=16 on a 0.5B model $\approx$ 10 MB):

```

du -sh logs/grpo_adapters/round_0001/adapters.safetensors
# e.g. 10M — if < 1 KB something went wrong

```

5. Verify Round-to-Round Adapter Loading

With adapter continuity enabled, each round resumes from the previous checkpoint. Watch for this log line at the start of round 2+:

```
# Round 1
[grpо_worker] Loading model: mlx-community/Qwen2.5-0.5B-Instruct-4bit
...
[grpо_worker] Result written to .../result_r0001_abc123.json

# Round 2
[grpо_worker] Loading adapter from round N-1: logs/grpo_adapters/round_0001
[grpо_worker] Adapter loaded – continuing from previous checkpoint
```

If the “Loading adapter” line is **missing**, `_last_adapter_path` in `train_async.py` was never set. Check that round 1 completed with `status == "success"`.

Manual adapter-load sanity check

```
# Run in the mlx-tune venv
/Volumes/ExternalSSD/train/mlx-tune/.venv/bin/python3 - <<'EOF'
from mlx_tune import FastLanguageModel

model, tok = FastLanguageModel.from_pretrained(
    "mlx-community/Qwen2.5-0.5B-Instruct-4bit", max_seq_length=512
)
model = FastLanguageModel.get_peft_model(model, r=16)
model.load_adapter("logs/grpo_adapters/round_0001")

from mlx.utils import tree_flatten
lora = [
    (k, v)
    for k, v in tree_flatten(model.model.trainable_parameters())
    if 'lora' in k
]
print(f"Loaded {len(lora)} LoRA tensors")
# Expected: Loaded 28 LoRA tensors (non-zero)
EOF
```

6. Verify Loss Behaviour in W&B

In W&B project `bochuxt7-iot/terminal-rl-simple`:

Chart	Healthy sign
grpo/loss vs grpo/step	Decreasing trend within a round
grpo/final_loss vs grpo/round	Decreasing trend across rounds
lora/trainable_params	Constant non-zero value every round
lora/applied	Always 1

If grpo/loss is flat or increasing:

- Learning rate too high -> try `GRPO_LR=1e-7`
- Too few generations -> advantages near zero -> try `GRPO_NUM_GEN=8`

- All rewards identical -> zero advantage signal -> check reward function

7. Diagnose a Failed Round

The result JSON is always written to logs/grpo_jobs/:

```
# Most recent result
ls -t logs/grpo_jobs/result_*.json | head -1 | xargs cat
{
  "status": "error",
  "adapter_path": "",
  "step_losses": [],
  "error": "Model load failed: ..."
}
```

Common errors:

Error message	Fix
mlx_tune not available	Wrong Python — check MLX_TUNE_PYTHON points to the venv
Model load failed	POLICY_MODEL_PATH is wrong or model not downloaded
Adapter load failed: FileNotFoundError	Previous adapter dir deleted or path changed
Adapter load failed: rank mismatch	GRPO_LORA_RANK changed between rounds — keep it constant
Worker exited with code 1	Check stderr in training log; usually import error or OOM
Worker timed out after 3600s	Model too large or GRPO_MAX_STEPS too high

8. Run the Full Unit Test Suite

Unit tests mock the subprocess and verify all contracts without a real model:

```
python3 -m pytest simple_rl/tests/test_mlx_grpo_bridge.py -m unit -v
```

20 tests cover:

- Dataset conversion (GRPOBatch -> prompt/answer format)
 - Subprocess command construction and job JSON fields
 - Stub mode when POLICY_MODEL_PATH is empty
 - Timeout and error handling
 - Worker script syntax validity
 - Reward function float parsing
 - _log_lora_verification() dict structure and W&B logging
 - wandb.define_metric x-axis setup for loss charts
 - step_losses structure and monotonically increasing steps
-

Quick Reference: Key Files

File	Role
<code>simple_rl/grpo_worker.py</code>	Runs under mlx-tune venv; loads model + adapter, trains, saves
<code>simple_rl/mlx_grpo_bridge.py</code>	Serialises job JSON, launches subprocess, reads result
<code>simple_rl/train_async.py</code>	Tracks <code>_last_adapter_path</code> ; passes it to each round
<code>logs/grpo_jobs/job_r*.json</code>	Input to each worker subprocess
<code>logs/grpo_jobs/result_r*.json</code>	Output from each worker subprocess
<code>logs/grpo_adapters/round_*/</code>	Saved LoRA weights per round
<code>logs/grpo_batch_r*.jsonl</code>	Rollout data snapshot per round